

NestJS Practical Test: Employee Management System (EMS)

Overview

Develop a RESTful API for an Employee Management System (EMS) to manage admin and employee records, attendance tracking, and generate punch-in/punch-out reports. The system treats Admin and Employee as separate entities, with distinct roles for admins (Super Admin and Admin) to differentiate privileges. It integrates MongoDB for data persistence and Redis for caching to optimize performance. A key feature allows admins to download a report of employees' punch-in/punch-out data.

Requirements

1. Setup

- Initialize a new NestJS project using the Nest CLI.
- Integrate MongoDB.
- Configure Redis for caching.
- Structure the project according to NestJS conventions, utilizing modules, services, and controllers.

2. Entities and DTOs

- **Admin Entity:**
 - Unique identifier (auto-generated).
 - Name (string, required).
 - Email (string, required, unique).
 - Password (string, required, hashed).
 - Role (enum: SUPER_ADMIN or ADMIN, required).
- **Employee Entity:**
 - Unique identifier (auto-generated).
 - Name (string, required).
 - Email (string, required, unique).
 - Password (string, required, hashed).
- **Attendance Entity:**
 - Unique identifier (auto-generated).
 - Employee reference (reference to Employee entity).
 - Punch-in timestamp (date, required).
 - Punch-out timestamp (date, optional).

- **DTOs:**
 - Create and update DTOs for admins, including fields for name, email, password, and role (SUPER_ADMIN or ADMIN).
 - Create and update DTOs for employees, including fields for name, email, and password.
 - Create and update DTOs for attendance, including employee reference and timestamps.
 - Use class-validator to enforce input validation for all DTOs.

3. Service Layer

- **AdminService:**
 - Create a new admin with validated data, assigning either SUPER_ADMIN or ADMIN role.
 - Retrieve all admins (Super Admin only).
 - Retrieve a specific admin by ID (self or Super Admin).
 - Update an admin's details by ID (self or Super Admin).
 - Delete an admin by ID (Super Admin only).
- **EmployeeService:**
 - Create a new employee with validated data (Super Admin or Admin).
 - Retrieve all employees (Super Admin or Admin).
 - Retrieve a specific employee by ID (admin or self).
 - Update an employee's details by ID (admin or self).
 - Delete an employee by ID (Super Admin or Admin).
- **AttendanceService:**
 - Record a punch-in event for an employee.
 - Record a punch-out event, updating the corresponding punch-in record.
 - Retrieve all attendance records, with optional filtering by employee or date range (Super Admin or Admin).
 - Generate a report of employees' punch-in/punch-out data, supporting downloadable formats (e.g., CSV or PDF) (Super Admin or Admin).
- **Redis Integration:**
 - Cache admin and employee data to reduce database queries.
 - Implement cache invalidation on admin or employee updates or deletions.
 - Fallback to database queries if Redis is unavailable.

4. Controller Layer

- **AuthController:**
 - Handle login for admins (Super Admin or Admin) and employees, issuing JWTs upon successful authentication.
- **AdminController:**
 - Create a new admin with role assignment (Super Admin only).
 - Retrieve all admins (Super Admin only).
 - Retrieve a specific admin by ID (self or Super Admin).
 - Update an admin's details by ID (self or Super Admin).
 - Delete an admin by ID (Super Admin only).
- **EmployeeController:**
 - Create a new employee (Super Admin or Admin).
 - Retrieve all employees (Super Admin or Admin).
 - Retrieve a specific employee by ID (admin or self).
 - Update an employee's details by ID (admin or self).
 - Delete an employee by ID (Super Admin or Admin).
- **AttendanceController:**
 - Record punch-in for an employee.
 - Record punch-out for an employee.
 - Retrieve attendance records (Super Admin or Admin, with optional filters).
 - Download a report of employees' punch-in/punch-out data (Super Admin or Admin), with options for filtering by date range or employee.

5. Error Handling

- Implement error handling for:
 - Admin, employee, or attendance record not found.
 - Validation errors (e.g., missing required fields, invalid email format).
 - Duplicate email addresses for admins or employees.
 - Unauthorized access (e.g., employees accessing admin-only endpoints, Admin accessing Super Admin-only endpoints, non-self admin/employee updates).
 - Errors during report generation (e.g., no data available for the specified filters).

6. Testing (Optional - Good to have)

- Write unit tests for AdminService, EmployeeService, and AttendanceService using Jest.

- Write unit tests for AuthController, AdminController, EmployeeController, and AttendanceController.
- Ensure tests cover all functionalities, including edge cases (e.g., invalid IDs, duplicate emails, unauthorized access, empty report data).
- Test role-based access control (e.g., Super Admin vs. Admin permissions).
- Test report generation for various scenarios, including filtered and unfiltered data.

7. Documentation

- Document all API endpoints using Swagger.
- Include details about request/response formats, including examples for admin/employee creation, attendance recording, authentication, and report downloading.
- Specify role-based access restrictions (Super Admin vs. Admin) in the documentation.
- Ensure documentation is clear and accessible via a dedicated endpoint.

8. Bonus

- Implement pagination for retrieving all admins, employees, and attendance records.
- Add filtering options for employees by department (assuming a department field is added to Employee entity) or attendance by date range.
- Support soft deletion for admins and employees to maintain data integrity.
- Allow report customization (e.g., selecting specific fields or date ranges for the punch-in/punch-out report).

Submission

The candidate should submit:

- A link to the GitHub repository with the completed project.
- Instructions for setting up and running the project locally, including MongoDB and Redis configuration.
- A brief explanation of the approach, design decisions (especially regarding separate Admin and Employee entities, admin roles, and report generation), and any challenges encountered.

Evaluation Criteria

- **Code Quality:** Adherence to NestJS best practices, clean and organized code structure.
- **TypeScript Proficiency:** Effective use of TypeScript for type safety and interfaces.
- **NestJS Knowledge:** Proper implementation of modules, services, controllers, and dependency injection.

- **MongoDB Integration:** Correct use of Mongoose for data modeling and relationships with separate Admin and Employee entities.
- **Redis Integration:** Effective caching strategy for admin and employee data, with proper invalidation and fallback mechanisms.
- **Admin Roles:** Robust implementation of role-based access control for Super Admin and Admin.
- **Report Functionality:** Effective implementation of the punch-in/punch-out report download feature, including filtering and format support.
- **Testing:** Comprehensive unit tests covering services, controllers, role-based access, and report generation, including edge cases.
- **Documentation:** Clear, detailed, and complete API documentation using Swagger, with explicit mention of role-based restrictions.